

User Acceptance Testing (UAT) Document

Caixa ERP - Loan Management System

Version: 1.0 **Date:** 2026-01-XX **Prepared by:** QA Team **Project:** Caixa ERP Development

Table of Contents

- 1. [System Overview](#)
- 2. [User Roles & Permissions](#)
- 3. [Test Environment Setup](#)
- 4. [Module 1: Loan Management](#)
- 5. [Module 2: Wallet Management](#)
- 6. [Module 3: Wallet Withdrawal](#)
- 7. [Module 4: Loan Repayment](#)
- 8. [Module 5: REST API Endpoints](#)
- 9. [Module 6: Reports & Dashboards](#)
- 10. [Integration Testing](#)
- 11. [Performance Testing](#)
- 12. [Test Sign-off](#)

System Overview

Purpose

Caixa ERP is a comprehensive loan management system integrated with Banking-as-a-Service (BaaS) for wallet operations. The system manages the complete loan lifecycle from application to closure, including wallet creation, disbursement, repayment, and withdrawal requests.

Key Modules

- 1. **Loan Management** (`dev_loan_management_caixa`) - Core loan processing
- 2. **Wallet Withdrawal** (`loan_wallet_withdrawal`) - Customer withdrawal requests
- 3. **REST API** (`rest_api_odoo`) - External API integration
- 4. **Custom API** (`custom_api`) - Additional API endpoints

Technology Stack

- **Framework:** Odoo 18
- **Database:** PostgreSQL
- **External Integration:** BaaS API (Banking as a Service)
- **Authentication:** API Key-based

User Roles & Permissions

Role	Permissions	Access Level
System Administrator	Full access to all modules	All CRUD operations
Loan Manager	Loan approval, disbursement, closure	Read/Write loans
Accountant	Financial operations, journal entries	Read/Write financial data
Loan Officer	Loan creation, review, customer interaction	Read/Write loans (limited)
Customer	View own loans, request withdrawals	Read-only (via API)

Test Environment Setup

Prerequisites

- ☒ Odoo 18 instance running
- ☒ PostgreSQL database configured
- ☒ BaaS API credentials configured in System Parameters:
 - `baas.base_url`
 - `baas.client_id`
 - `baas.client_secret`
- ☒ Test customer records with BVN data
- ☒ API keys generated for test users
- ☒ Test bank account details

Test Data Requirements

- ☐ At least 5 test customers (partners)
- ☐ 3 loan types configured
- ☐ 2 loan officers
- ☐ 1 loan manager
- ☐ 1 accountant
- ☐ Test currency (NGN)

Module 1: Loan Management

TC-1.1: Create New Loan Application

Objective: Verify loan application creation workflow

Preconditions:

- User logged in as Loan Officer

- Customer exists with `is_allow_loan = True`
- Loan type configured

Test Steps:

1. Navigate to **Loans** → **Loans** → **Create**
2. Fill in loan details:
3. Customer: Select test customer
4. Loan Type: Select configured loan type
5. Loan Amount: 100,000
6. Loan Term: 5 months
7. Interest Rate: 10%
8. Interest Mode: Reducing
9. Request Date: Today
10. Click **Save**
11. Verify loan is created in **Draft** state
12. Verify loan reference number is auto-generated (format: `LN/YYYY/####`)

Expected Results:

- ☐ Loan created successfully
- ☐ State = "Draft"
- ☐ Reference number generated
- ☐ All fields saved correctly
- ☐ Customer can be selected only if `is_allow_loan = True`

Test Data:

Customer: John Doe
Loan Amount: 100,000 NGN
Loan Term: 5 months
Interest Rate: 10%
Interest Mode: Reducing

TC-1.2: Submit Loan for Review

Objective: Verify loan submission workflow

Preconditions:

- Loan exists in Draft state (from TC-1.1)

Test Steps:

1. Open the loan record
2. Click **Submit for Review**
3. Verify state changes to **Review**
4. Verify loan is visible to Loan Manager

Expected Results:

- ☐ State = "Review"
- ☐ Loan appears in Loan Manager's review queue

- [] Cannot edit loan details (read-only)
-

TC-1.3: Loan Review & Approval

Objective: Verify loan review and approval process

Preconditions:

- Loan in Review state (from TC-1.2)
- User logged in as Loan Manager

Test Steps:

1. Navigate to loan record
2. Review loan details
3. Click **Approve** button
4. Fill in approval reason
5. Click **Confirm**
6. Verify state changes to **Confirmed**

Expected Results:

- [] State = "Confirmed"
 - [] Approval reason saved
 - [] Approved by field populated with current user
 - [] Approval date recorded
-

TC-1.4: Final Approval

Objective: Verify final approval workflow

Preconditions:

- Loan in Confirmed state (from TC-1.3)

Test Steps:

1. Open loan record
2. Click **Submit for Final Approval**
3. Fill in final approval reason
4. Click **Confirm**
5. Verify state changes to **Final Approval**

Expected Results:

- [] State = "Final Approval"
 - [] Final approval reason saved
 - [] Cannot proceed to disbursement without final approval
-

TC-1.5: Loan Disbursement

Objective: Verify loan disbursement to customer wallet

Preconditions:

- Loan in Final Approval state
- Customer has wallet account number
- BaaS API accessible

Test Steps:

1. Open loan record
2. Click **Disburse Loan**
3. Select disbursement date
4. Click **Confirm**
5. Verify BaaS wallet credit transaction
6. Verify state changes to **Disbursed** then **Open**
7. Verify installment schedule is generated

Expected Results:

- [] State = "Open"
- [] Disbursement date recorded
- [] Installment schedule created (5 installments for 5-month loan)
- [] Each installment shows:
 - Principal amount
 - Interest amount
 - Daily interest (interest / 30)
 - Total EMI
 - Opening balance
 - Closing balance
- [] BaaS transaction ID recorded (if available)

Validation:

- [] Installment dates are 30 days apart (not calendar months)
 - [] Principal reduces each period (reducing balance)
 - [] Interest calculated correctly on remaining balance
 - [] Total of all installments = Loan Amount + Total Interest
-

TC-1.6: Loan Rejection

Objective: Verify loan rejection workflow

Preconditions:

- Loan in Review or Confirmed state

Test Steps:

1. Open loan record
2. Click **Reject** button
3. Fill in rejection reason (mandatory)
4. Click **Confirm**
5. Verify state changes to **Rejected**

Expected Results:

- [] State = "Rejected"
 - [] Rejection reason saved
 - [] Rejected by field populated
 - [] Loan cannot be edited or approved
-

TC-1.7: Loan Cancellation

Objective: Verify loan cancellation

Preconditions:

- Loan in Draft state

Test Steps:

1. Open loan record
2. Click **Cancel** button
3. Verify state changes to **Cancel**

Expected Results:

- [] State = "Cancel"
 - [] Loan cannot be submitted
-

TC-1.8: Loan Restructure

Objective: Verify loan restructuring functionality

Preconditions:

- Loan in Open state with at least 1 unpaid installment

Test Steps:

1. Open loan record
2. Click **Restructure Loan**
3. Modify loan term or interest rate
4. Fill in restructure reason
5. Click **Confirm**
6. Verify new installment schedule generated

Expected Results:

- [] New installment schedule created
 - [] Old installments marked appropriately
 - [] Restructure reason recorded
 - [] Loan amount remains same (unless specified)
-

TC-1.9: Loan Closure

Objective: Verify loan closure process

Preconditions:

- Loan in Open state
- All installments paid OR early closure requested

Test Steps:

1. Open loan record
2. Click **Close Loan**
3. Fill in closure reason
4. Verify outstanding balance calculation
5. Click **Confirm**
6. Verify state changes to **Closed**

Expected Results:

- [] State = "Closed"
 - [] Closure reason saved
 - [] Outstanding balance = 0
 - [] Loan cannot be reopened
-

Module 2: Wallet Management

TC-2.1: Create Tier One Wallet via API

Objective: Verify wallet creation through BaaS integration

Preconditions:

- BaaS API configured
- Valid BVN data available
- API key generated

Test Steps:

1. Send POST request to `/wallet/create-tier-one`
2. Headers:
3. `api-key` : Valid API key
4. `db` : Database name
5. `Content-Type` : application/json
6. Request body:

```
{
  "bvn": "31035529413",
  "firstname": "John",
  "lastname": "Doe",
  "phone": "3103452948",
  "dob": "1990-01-15",
  "partner_id": 123
}
```

1. Verify response

Expected Results:

- [] HTTP 200 response
- [] Response contains:

```
{
  "success": true,
  "account_number": "4000039258",
  "message": "success",
  "wallet_tier": "tier_1",
  "errors": []
}
```

- [] Customer record updated with wallet account number
- [] Wallet tier = "tier_1"
- [] Wallet status = "active"

Error Scenarios:

- [] Invalid BVN → Error message returned
 - [] Missing required fields → 400 Bad Request
 - [] Invalid API key → 401 Unauthorized
 - [] BaaS API failure → Error message with details
-

TC-2.2: Check Wallet Balance

Objective: Verify wallet balance retrieval

Preconditions:

- Customer has active wallet account number

Test Steps:

1. Navigate to customer record
2. View wallet account number field
3. Verify balance is displayed (if available)
4. Test via API if endpoint exists

Expected Results:

- [] Wallet account number visible
 - [] Balance displayed correctly
 - [] Currency shown (NGN)
-

Module 3: Wallet Withdrawal

TC-3.1: Create Withdrawal Request (Draft)

Objective: Verify withdrawal request creation

Preconditions:

- Customer has active loan in "Disbursed" or "Open" state
- Customer has wallet account number

Test Steps:

1. Navigate to **Accounting** → **Loan Wallet Withdrawals** → **Create**
2. Fill in withdrawal details:
3. Customer: Select customer with wallet
4. Related Loan: Select active loan (optional)
5. Withdrawal Amount: 50,000
6. Bank Name: "Access Bank"
7. Account Number: "1234567890"
8. Account Name: "John Doe"
9. Click **Save**
10. Verify state = "Draft"

Expected Results:

- ☐ Withdrawal request created
 - ☐ State = "Draft"
 - ☐ Reference number auto-generated (format: WDR/YYYY/####)
 - ☐ Wallet account number auto-populated from customer
 - ☐ All fields saved correctly
 - ☐ Can edit all fields in Draft state
-

TC-3.2: Submit Withdrawal for Approval

Objective: Verify withdrawal submission

Preconditions:

- Withdrawal request in Draft state (from TC-3.1)

Test Steps:

1. Open withdrawal request
2. Click **Submit** button
3. Verify state changes

Expected Results:

- ☐ State = "Submitted for Approval"
 - ☐ Cannot edit amount or bank details
 - ☐ Request visible to approvers
-

TC-3.3: Approve Withdrawal Request

Objective: Verify withdrawal approval

Preconditions:

- Withdrawal request in Submitted state

- User logged in as Accountant or Manager

Test Steps:

1. Open withdrawal request
2. Click **Approve** button
3. Verify state changes to **Approved**

Expected Results:

- ☐ State = "Approved"
 - ☐ Approval date recorded
 - ☐ Approved by field populated
 - ☐ Transaction ID field becomes visible (but not required yet)
-

TC-3.4: Complete Withdrawal (Mark as Done)

Objective: Verify withdrawal completion with transaction ID

Preconditions:

- Withdrawal request in Approved state
- Money transferred outside Odoo

Test Steps:

1. Open withdrawal request
2. Enter Transaction ID (mandatory field)
3. Click **Mark as Done**
4. Verify state changes

Expected Results:

- ☐ State = "Done"
- ☐ Transaction ID is required (cannot proceed without it)
- ☐ Completion date recorded
- ☐ Completed by field populated
- ☐ Request cannot be edited

Validation:

- ☐ Cannot mark as Done without Transaction ID
 - ☐ Error message shown if Transaction ID is empty
-

TC-3.5: Reject Withdrawal Request

Objective: Verify withdrawal rejection

Preconditions:

- Withdrawal request in Submitted or Approved state

Test Steps:

1. Open withdrawal request

2. Click **Reject** button
3. Fill in rejection reason (mandatory)
4. Click **Confirm**
5. Verify state changes

Expected Results:

- [] State = "Rejected"
 - [] Rejection reason saved
 - [] Request cannot be approved after rejection
-

TC-3.6: Cancel Withdrawal Request

Objective: Verify withdrawal cancellation

Preconditions:

- Withdrawal request in Draft state

Test Steps:

1. Open withdrawal request
2. Click **Cancel** button
3. Verify state changes

Expected Results:

- [] State = "Cancelled"
 - [] Request cannot be submitted
-

TC-3.7: View Withdrawal History per Customer

Objective: Verify withdrawal tracking

Preconditions:

- Multiple withdrawal requests for same customer

Test Steps:

1. Navigate to customer record
2. View "Withdrawal Requests" tab
3. Verify all withdrawals listed
4. Check withdrawal count

Expected Results:

- [] All withdrawal requests visible
 - [] Count displayed correctly
 - [] Can filter by state
 - [] Can open individual withdrawal records
-

TC-3.8: Create Withdrawal via API

Objective: Verify API endpoint for withdrawal creation

Preconditions:

- Valid API key
- Customer with wallet account number

Test Steps:

1. Send POST request to `/wallet_withdrawal`
2. Headers:
3. `api-key` : Valid API key
4. `db` : Database name
5. `Content-Type` : application/json
6. Request body:

```
{
  "partner_id": 123,
  "loan_id": 5,
  "withdrawal_amount": 50000,
  "bank_name": "Access Bank",
  "account_number": "1234567890",
  "account_name": "John Doe"
}
```

1. Verify response

Expected Results:

- [] HTTP 200 response
- [] Withdrawal request created
- [] Response contains withdrawal ID and reference
- [] Wallet balance validated before creation
- [] Error if insufficient balance

Module 4: Loan Repayment

TC-4.1: View Repayment Schedule

Objective: Verify installment schedule display

Preconditions:

- Loan in Open state with generated installments

Test Steps:

1. Open loan record
2. Navigate to "Installments" tab
3. Review installment schedule
4. Verify columns displayed

Expected Results:

- ☐ All installments listed
 - ☐ Columns visible:
 - Installment Name
 - Date (Due Date)
 - State (Unpaid/Paid)
 - Principal Amount
 - Interest Amount
 - **Daily Interest** (Interest/Day)
 - Total Amount (EMI)
 - Opening Balance
 - Closing Balance
 - Payment Date
 - Penalty Amount
 - ☐ Installments ordered by date (ascending)
 - ☐ Daily Interest = Interest / 30
-

TC-4.2: Pay Installment (Full Period)

Objective: Verify payment of installment on due date

Preconditions:

- Loan in Open state
- First installment unpaid
- Payment date = Installment due date

Test Steps:

1. Open loan record
2. Navigate to Installments tab
3. Select first unpaid installment
4. Click **Pay** button
5. Review payment wizard:
6. Days Elapsed: 30
7. Principal Amount: (e.g., 20,000)
8. Interest (Pro-rated): (e.g., 10,000) - Full interest
9. Penalty: 0
10. Total Amount: (e.g., 30,000)
11. Enter paid amount (should match total)
12. Click **Pay Installment**
13. Verify installment marked as paid

Expected Results:

- ☐ Installment state = "Paid"
- ☐ Payment date = Today
- ☐ Journal entry created:
- Principal debit
- Interest debit
- Cash/Bank credit

- [] Opening balance of next installment updated
 - [] Loan remaining amount reduced
-

TC-4.3: Pay Installment Early (Pro-rated Interest)

Objective: Verify pro-rated interest calculation for early payment

Preconditions:

- Loan disbursed on Day 1
- First installment due on Day 30
- Today = Day 5 (5 days elapsed)

Test Steps:

1. Open loan record
2. Navigate to Installments tab
3. Select first unpaid installment
4. Click **Pay** button
5. Review payment wizard:
6. Days Elapsed: 5
7. Principal Amount: (e.g., 20,000)
8. Interest (Pro-rated): (e.g., 1,666.67) - 5 days $\times$ daily_interest
9. Penalty: 0
10. Total Amount: (e.g., 21,666.67)
11. Verify calculation:
12. Daily Interest = Full Interest / 30
13. Pro-rated Interest = Daily Interest $\times$ 5
14. Enter paid amount
15. Click **Pay Installment**

Expected Results:

- [] Days Elapsed = 5 (not 30)
- [] Pro-rated interest calculated correctly
- [] Total amount = Principal + Pro-rated Interest
- [] Installment marked as paid
- [] Journal entry uses pro-rated interest (not full interest)
- [] Next installment opening balance updated correctly

Calculation Example:

```
Loan: 100,000 NGN
Term: 5 months
Interest Rate: 10% (reducing)
First Installment:
- Principal: 20,000
- Full Interest: 10,000 (for 30 days)
- Daily Interest: 333.33
- Days Elapsed: 5
- Pro-rated Interest:  $333.33 \times 5 = 1,666.65$ 
- Total to Pay:  $20,000 + 1,666.65 = 21,666.65$ 
```

TC-4.4: Pay Installment with Penalty

Objective: Verify penalty calculation and payment

Preconditions:

- Installment overdue (past due date)
- Penalty configured

Test Steps:

1. Open overdue installment
2. Click **Pay** button
3. Review payment wizard:
4. Penalty Amount: (e.g., 500)
5. Total Amount: Principal + Interest + Penalty
6. Enter paid amount
7. Click **Pay Installment**

Expected Results:

- [] Penalty amount displayed
 - [] Total includes penalty
 - [] Journal entry includes penalty account
 - [] Installment marked as paid
-

TC-4.5: Pay Multiple Installments

Objective: Verify payment of multiple installments at once

Preconditions:

- Loan with multiple unpaid installments

Test Steps:

1. Open loan record
2. Navigate to Installments tab
3. Select multiple unpaid installments
4. Click **Pay Selected** (if available)
5. Review total amount
6. Confirm payment

Expected Results:

- [] All selected installments marked as paid
 - [] Journal entries created for each
 - [] Total amount = Sum of all installments
-

TC-4.6: Repayment via API (Single Installment)

Objective: Verify API endpoint for installment payment

Preconditions:

- Loan in Open state
- Unpaid installments exist
- Customer has sufficient wallet balance
- Valid API key

Test Steps:

1. Send POST request to `/loans/{loan_id}/pay-installment`
2. Headers:
3. `api-key` : Valid API key
4. `db` : Database name
5. `Content-Type` : application/json
6. Request body (optional):

```
{
  "installment_id": 10
}
```

1. Verify response

Expected Results:

- [] HTTP 200 response
- [] Response contains:

```
{
  "success": true,
  "message": "Repayment successful",
  "data": {
    "installment_id": 10,
    "amount_paid": 21666.65,
    "proRatedInterest": 1666.65,
    "penalty": 0.0,
    "transaction_id": "TXN-BAAS-123456"
  }
}
```

- [] BaaS wallet debited
- [] Installment marked as paid in Odoo
- [] Pro-rated interest calculated automatically

Error Scenarios:

- [] No unpaid installments → Error message
- [] Insufficient wallet balance → BaaS error returned
- [] Invalid installment_id → 404 Not Found
- [] Installment out of sequence → Error (must pay oldest first)

TC-4.7: Get Repayment Schedule via API

Objective: Verify API endpoint for schedule retrieval

Preconditions:

- Loan in Open state
- Installments generated
- Valid API key

Test Steps:

1. Send GET request to `/loans/{loan_id}/repayment-schedule`
2. Headers:
3. `api-key` : Valid API key
4. `db` : Database name
5. Optional query params:
6. `status` : paid/unpaid/all
7. `fields` : comma-separated field list

Expected Results:

- [] HTTP 200 response
- [] Response contains:

```
{
  "success": true,
  "loan_id": 5,
  "loan_name": "LN/2026/00005",
  "loan_amount": 100000.0,
  "summary": {
    "total_installments": 5,
    "paid_count": 0,
    "unpaid_count": 5,
    "total_principal": 100000.0,
    "total_interest": 40000.0,
    "total_emi": 140000.0,
    "total_penalty": 0.0,
    "outstanding_balance": 100000.0
  },
  "next_due": {
    "id": 10,
    "name": "INS/001",
    "date": "2026-03-12",
    "state": "unpaid",
    "amount": 20000.0,
    "interest": 10000.0,
    "daily_interest": 333.33,
    "total_amount": 30000.0,
    ...
  },
  "schedule": [...]
}
```

- ☐ All installments include `daily_interest` field
 - ☐ Summary statistics accurate
 - ☐ Next due installment highlighted
-

Module 5: REST API Endpoints

TC-5.1: API Authentication

Objective: Verify API key authentication

Test Steps:

1. Send request without `api-key` header
2. Verify response

Expected Results:

- ☐ HTTP 401 Unauthorized
- ☐ Error message: "No API Key Provided"

Test Steps:

1. Send request with invalid `api-key`
2. Verify response

Expected Results:

- ☐ HTTP 401 Unauthorized
 - ☐ Error message: "Invalid API Key"
-

TC-5.2: Get Loan Details

Endpoint: `GET /loans/{loan_id}`

Test Steps:

1. Send GET request with valid API key
2. Verify response

Expected Results:

- ☐ HTTP 200
 - ☐ Loan details returned in JSON
 - ☐ All relevant fields included
-

TC-5.3: Get Loan Stage/Status

Endpoint: `GET /loans/{loan_id}/stage`

Test Steps:

1. Send GET request

2. Verify response

Expected Results:

- [] HTTP 200
 - [] Current loan state returned
 - [] Status description included
-

TC-5.4: Get Agreement Data

Endpoint: GET /loans/{loan_id}/agreement-data

Test Steps:

1. Send GET request
2. Verify response

Expected Results:

- [] HTTP 200
 - [] Agreement template data returned
 - [] Customer and loan details included
-

TC-5.5: Log Agreement Email

Endpoint: POST /loans/{loan_id}/log-agreement-email

Test Steps:

1. Send POST request with email details
2. Verify response

Expected Results:

- [] HTTP 200
 - [] Email logged in loan record
 - [] Activity created
-

TC-5.6: Customer Response

Endpoint: POST /loans/{loan_id}/customer-response

Test Steps:

1. Send POST request with customer response
2. Verify response

Expected Results:

- [] HTTP 200
 - [] Response recorded
 - [] Loan state updated if applicable
-

TC-5.7: Pay Multiple Installments via API

Endpoint: POST /loans/{loan_id}/pay-multi-installments

Test Steps:

1. Send POST request with installment IDs
2. Verify response

Expected Results:

- [] HTTP 200
 - [] All installments processed
 - [] Total amount calculated correctly
-

TC-5.8: Clear Loan (Full Payment)

Endpoint: POST /loans/{loan_id}/clear-loan

Test Steps:

1. Send POST request
2. Verify response

Expected Results:

- [] HTTP 200
 - [] All unpaid installments marked as paid
 - [] Loan state = "Closed"
 - [] Outstanding balance = 0
-

TC-5.9: Get Withdrawal Status

Endpoint: GET /withdrawals/{withdrawal_id}/status

Test Steps:

1. Send GET request
2. Verify response

Expected Results:

- [] HTTP 200
 - [] Withdrawal status returned
 - [] Current state included
-

TC-5.10: BaaS Webhook Receiver

Endpoint: POST /api/baas/webhook

Test Steps:

1. Send POST request with BaaS webhook payload
2. Verify processing

Expected Results:

- ☐ HTTP 200
 - ☐ Webhook data processed
 - ☐ Related records updated (if applicable)
-

Module 6: Reports & Dashboards

TC-6.1: Loan Summary Report

Objective: Verify loan summary report generation

Test Steps:

1. Navigate to **Loans → Reports → Loan Summary**
2. Select date range
3. Generate report
4. Verify content

Expected Results:

- ☐ Report generated successfully
 - ☐ All loans in date range included
 - ☐ Totals calculated correctly
 - ☐ PDF format available
-

TC-6.2: Installment Summary Report

Objective: Verify installment summary report

Test Steps:

1. Navigate to **Loans → Reports → Installment Summary**
2. Select loan or date range
3. Generate report
4. Verify content

Expected Results:

- ☐ Report generated
 - ☐ All installments listed
 - ☐ Paid/Unpaid status shown
 - ☐ Totals accurate
-

TC-6.3: Interest Certificate

Objective: Verify interest certificate generation

Test Steps:

1. Navigate to loan record
2. Click **Generate Interest Certificate**

3. Verify certificate

Expected Results:

- ☐ Certificate generated
 - ☐ Interest details accurate
 - ☐ Customer information correct
 - ☐ PDF format available
-

TC-6.4: Loan Dashboard

Objective: Verify loan dashboard display

Test Steps:

1. Navigate to **Loans** → **Dashboard**
2. Review KPIs and charts
3. Verify data accuracy

Expected Results:

- ☐ Dashboard loads
 - ☐ KPIs displayed:
 - Total Loans
 - Active Loans
 - Total Disbursed
 - Outstanding Amount
 - Overdue Installments
 - ☐ Charts render correctly
 - ☐ Data filters work
-

Integration Testing

TC-INT-1: End-to-End Loan Lifecycle

Objective: Verify complete loan process from application to closure

Test Steps:

1. **Create Customer & Wallet:**
2. Create customer via API
3. Create Tier One wallet via API
4. Verify wallet account number assigned
5. **Create Loan Application:**
6. Create loan in Draft state
7. Submit for review
8. Approve loan
9. Final approval

10. Disburse to wallet
11. **Verify Disbursement:**
12. Check wallet balance increased
13. Verify installment schedule generated
14. Confirm daily interest calculated
15. **Customer Withdrawal:**
16. Create withdrawal request
17. Submit for approval
18. Approve withdrawal
19. Complete with transaction ID
20. **Loan Repayment:**
21. Pay first installment (early - Day 5)
22. Verify pro-rated interest
23. Pay remaining installments
24. Verify journal entries
25. **Loan Closure:**
26. Close loan after all payments
27. Verify final state

Expected Results:

- ☐ All steps completed successfully
 - ☐ Data consistency maintained
 - ☐ Journal entries accurate
 - ☐ Wallet balance correct at each stage
-

TC-INT-2: BaaS Integration

Objective: Verify BaaS API integration

Test Scenarios:

1. **Wallet Creation:**
2. ☐ Success case
3. ☐ Invalid BVN handling
4. ☐ Network failure handling
5. **Wallet Debit (Repayment):**
6. ☐ Success case
7. ☐ Insufficient balance
8. ☐ Invalid account number

9. **Wallet Credit (Disbursement):**

- 10. ☐ Success case
- 11. ☐ Invalid account number
- 12. ☐ Transaction ID recording

Expected Results:

- ☐ All BaaS calls logged
 - ☐ Error handling robust
 - ☐ Transaction IDs recorded
 - ☐ Retry logic works (if implemented)
-

Performance Testing

TC-PERF-1: Load Testing

Objective: Verify system performance under load

Test Scenarios:

- 1. **API Endpoints:**
 - 2. ☐ 100 concurrent requests to repayment schedule endpoint
 - 3. ☐ Response time < 2 seconds
 - 4. ☐ No errors
 - 5. **Loan Processing:**
 - 6. ☐ Create 50 loans simultaneously
 - 7. ☐ All processed successfully
 - 8. ☐ No data corruption
 - 9. **Report Generation:**
 - 10. ☐ Generate report for 1000 loans
 - 11. ☐ Completion time < 30 seconds
-

Test Sign-off

Test Execution Summary

Module	Test Cases	Passed	Failed	Blocked	Status
Loan Management	9				
Wallet Management	2				
Wallet Withdrawal	8				
Loan Repayment	7				
REST API	10				
Reports & Dashboards	4				
Integration	2				
Performance	1				
TOTAL	43				

Sign-off

Tested By:

- Name: ____
- Date: ____
- Signature: ____

Approved By:

- Name: ____
- Date: ____
- Signature: ____

Project Manager:

- Name: ____
- Date: ____
- Signature: ____

Appendix

A. Test Data Templates

Customer Template

Name: Test Customer {N}
Email: test{n}@example.com
Phone: +234801234567{n}

BVN: 3103552941{n}
is_allow_loan: True

Loan Template

Customer: Test Customer 1
Loan Type: Personal Loan
Loan Amount: 100,000
Loan Term: 5 months
Interest Rate: 10%
Interest Mode: Reducing
None Interest Month: 0

B. API Test Collection

Postman collection available at: /rest_api_odoo/Postman Collections/Odoo REST
Api.postman_collection.json

C. Known Issues

Issue ID	Description	Severity	Status

D. Test Environment Details

- **Odoo Version:** 18.0
- **Database:** PostgreSQL
- **Server URL:** [To be filled]
- **API Base URL:** [To be filled]
- **BaaS API URL:** [To be filled]

Document Version: 1.0 **Last Updated:** 2026-01-XX **Next Review:** [Date]